Content

이번 주차의 학습목표와 학습내용을 확인하세요.

01. 파이썬 언어의 기초와 컬렉션 자료형

파이썬 언어를 이해하고 컬렉션 자료형을 활용할 수 있다.

02. 함수, 변수, 모듈

함수, 변수, 모듈에 대해 이해하고 설명할 수 있다.

03. 객체지향 프로그래밍

객체지향 프로그래밍에 대해 이해하고 설명할 수 있다.

파이썬 언어의 기초

파이썬이란

파이썬의 특징

1991년 귀도 반 로섬(Guido van Rossum)에 의해 개발

AI, 빅데이터의 부상과 함께 최근 각광

문법이 쉽고 직관적

<출처: Jason E. Kaplan. https://gvanrossum.github.io//>

파이썬이란

파이썬의 특징

인터프리터 방식

컴파일 방식

스크립트 모드 지원

통합개발환경

- IDLE
- 주피터 노트북
- 파이참
- 비주얼 스튜디오

파이썬이란

파이썬의 특징

프로그래밍 언어 인기 순위

2024년 1월 기준

2024년 1월	2023년 1월	프로그래밍 언어	비율
1	1	Python	13.97%
2	2	C	11.44%
3	3	C++	9.96%
4	4	Java	7.87%
5	5	C#	7.16%
6	7	JavaScript	2.77%

<출처: https://www.tiobe.com/tiobe-index/>

파이썬이란

예약어

and	as	assert	break	class
continue	def	del	elif	else
except	finally	for	from	global
if	import	in	is	lambda
nonlocal	not	or	pass	raise
return	try	while	with	yield
False	None	True		

리터럴, 자료형, 변수

변수와 리터럴의 의미

$$x = 100$$

변수 리터럴

리터럴, 자료형, 변수

리터럴(Literal)과 자료형

분류	내장 자료형	리터럴			
수치	정수(int)	10	-30	0xfffe	073
	실수(float)	3.14		-0.45	123.032E-13
	복소수(complex)	complex(1,2)		1+2j	4+5j
	부울(bool)	True		False	
시퀀스	문자열(str)	'game'		"over"	"C"
	리스트(list)	[]		[0,1,2,3]	[0,'hello',3.14]
	튜플(tuple)	(0,1,2,3)		('hello','world','game')	
매핑	딕셔너리(dict)	{3.14: "phi",4.5: "score"}			
집합	집합(set, frozenset)	{1,2,3}		{'one','two','three'}	

리터럴, 자료형, 변수

변수(Variable)

```
number = 132                          # 변수 생성 및 사용 문장
number = number + 8                   # 변수 사용 문장
```

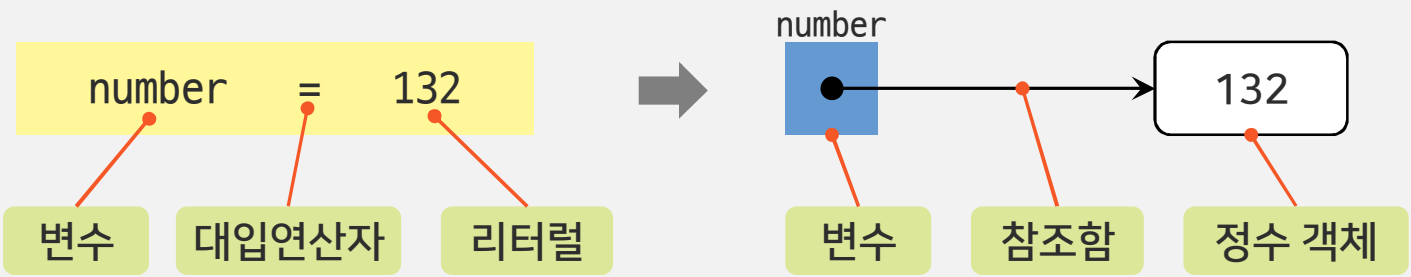
```
pi = 3.14                             # float 변수
comp = 1 + 2j                         # complex 변수
isValid = True                        # bool 변수
msg = ‘game over !!!’                 # str 변수
A = [0, 1, 1, 2, 3, 5, 8, 13]         # list 변수
```

리터럴, 자료형, 변수

변수(Variable)

파이썬에서는 모든 자료가 클래스로부터 만들어진 객체임

변수는 다른 객체를 참조하는 참조자 또는 포인터의 역할을 함



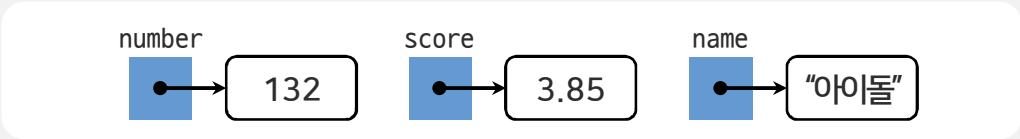
리터럴, 자료형, 변수

변수(Variable)

변수의 동작 이해

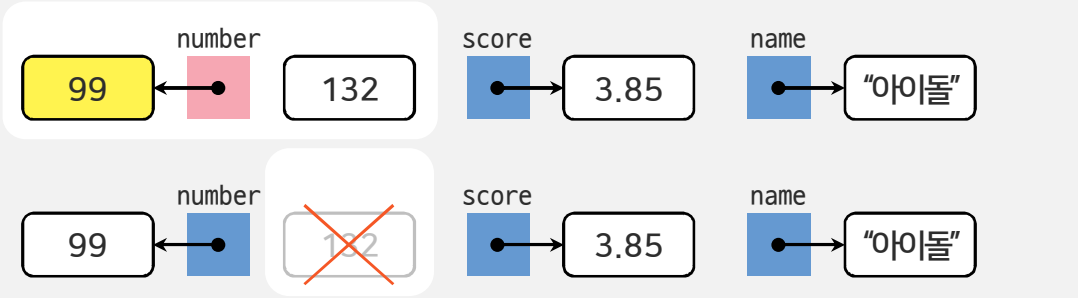
01

```
number = 132
score = 3.85
name = "아이돌"
```



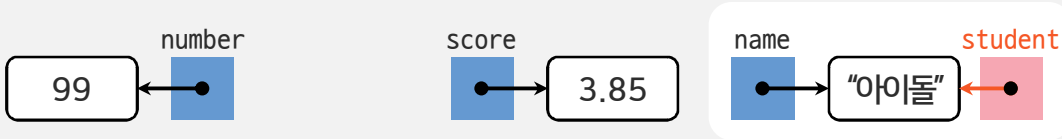
02

```
number = 99
```



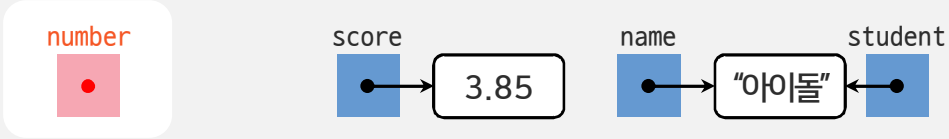
03

```
student = name
```



04

```
number = None
```



연산자

01 나눗셈 연산자

연산자 /	연산자 //
실수의 나눗셈 (결과가 실수)	정수 연산(floor division)

02 이항, 단항 연산자

이항 연산자 ** 추가	단항 연산자 ++, -- 미제공
예 3**7	예 x++ (x) x += 1 (o)

연산자

03 관계 연산자

>	<	>=	<=	==	!=

04 불리언 연산자

||	&&	!

05 in과 not in 연산자

```
'a' in 'banana'      # True
'seed' in 'banana'   # False
```

```
A = [0, 1, 1, 2, 3, 5, 8, 13]
if 3 in A :
while 4 in A :
```

함수호출과 입출력 함수

함수호출

```
y = sum (a, b)                          # 함수 sum 호출. 인수 2개
w = x * z + func(a, b, c)               # 함수 func 호출. 인수 3개
print(“game over”)                      # 화면출력함수 print 호출.
```

함수호출과 입출력 함수

키보드 입력 함수 input

```
name = input("당신의 이름을 입력하세요.")                    # 문자열
hobby = input("취미가 무엇입니까?")                          # 문자열
```

```
age = int( input("나이가 몇 살입니까?"))                     # 입력 문자열을 정수로 변환
score = float( input("평균 학점이 얼마입니까?"))             # 입력 문자열을 실수로 변환
```

함수호출과 입출력 함수

화면 출력함수 print

```
print("game over")
```

```
print("game", end="")
print("over")
```

출력할 때 활용되는 명령어

\\ Backslash(\)	\n Newline	\" Double quote(")
\' Single quote(')	\t Horizontal tab	

제어구조와 반복

분기(Branching): if, else, elif

```
if score >= 90 :
    print("합격입니다.")
    print("장학금 대상입니다.")
```

```
if score >= 90 :
    print("합격입니다.")
print("장학금 대상입니다.")
```

```
if value % 2 == 0 :
    print("짝수입니다.")
```

```
if value % 2 == 0 : print("짝수입니다.")        # 블록을 한 줄로 처리
else : print("홀수입니다.")                      # 블록을 한 줄로 처리
```

제어구조와 반복

분기(Branching): if, else, elif

```
if score >= 90 :
    grade = “A”
else :
    if score >= 80 :
        grade = “B”
    else :
        if score >= 70 :
            grade = “C”
        else :
            if score >= 60 :
                grade = “D”
            else :
                grade = “F”
```

```
if score >= 90 :
    grade = “A”
elif score >= 80 :
    grade = “B”
elif score >= 70 :
    grade = “C”
elif score >= 60 :
    grade = “D”
else :
    grade = “F”
```

제어구조와 반복

반복(Looping)

```
dan = int(input("구구단 단 입력: ")
for n in range( 2, 10, 1 ) :
     print("%2d x %2d = "% (dan, n), dan*n )
```

```
dan = int(input("구구단 단 입력: ")
n = 2
while n < 10 :
     print("%2d x %2d = "% (dan, n), dan*n )
     n += 1
```

제어구조와 반복

범위(Range)

```
for n in range(5) :                          # n: 0, 1, 2, 3, 4
for n in range(2, 10) :                      # n: 2, 3, …, 9
for n in range(10, 3, -2) :                  # n: 10, 8, 6, 4
```

```
for item in [12, 33, 52, 26, 99] :           # 리스트의 모든 항목에 대해 반복
     print( “값 =”, item)                    # 12, 33, 52, 26, 99 출력
```

```
for c in “Game Over !” :                     # 문자열의 각 문자에 대해
     print( “값 =”, c)                       # “G”, “a”, “m”, …, “!” 출력
```

제어구조와 반복

범위(Range)

```
mySet = set([12, 33, 52, 26, 99])
for e in mySet :
     print( “값 =”, e)
```

```
myDict = { ‘A’:12, ‘B’:33, ‘C’:52, ‘D’:26, ‘E’:99 }
for e in myDict :
     print( “키 =”, e)
     print( “값 =”, myDict[e])
```

컬렉션 자료형

컬렉션 자료형의 활용

문자열(str)

```
msg = ‘game over’
hi = “hello world”
sum = “예전엔” + hi + “이제는” + msg
print sum
```

C:\WINDOWS\system32\cmd.exe

```
예전엔 hello world 이제는 game over
```

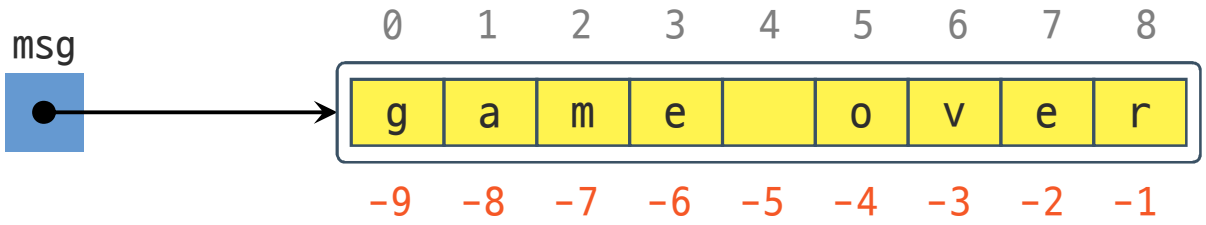
컬렉션 자료형의 활용

문자열(str)

```
print(msg, '의 첫 글자는', msg[0])
print(msg, '의 끝 글자는', msg[-1])
```

C:\WINDOWS\system32\cmd.exe

```
game over 의 첫 글자는 g
game over 의 끝 글자는 r
```



컬렉션 자료형의 활용

문자열(str)

```
hobby = “테니스”
age = 21
score = 4.5
msg1 = “당신의 학점은 %4.1f입니다” % score
msg2 = “취미=%s, 나이=%d, 학점=%f” % (hobby, age, score)
```

컬렉션 자료형의 활용

리스트(list)

```
big3 = []
lotto = [23, 34, 11, 42, 9]
big4 = [‘제이플라’, ‘도티’, ‘대도서관’, ‘보람튜브’]
```

```
print(“lotto[1] =”, lotto[1])
big4[2] = ‘블랙핑크’
```

```
big3.append(“알라딘”)
big3.append(“엘사”)
big3.append(“안나”)
```

컬렉션 자료형의 활용

리스트(list)

리스트의 메소드

메소드	설명
s.append(item)	• 항목 item을 리스트 s의 맨 뒤에 추가함
s.extend(lst)	• 리스트 lst를 s에 추가함
s.count(item)	• 리스트에서 항목 item의 개수를 세고 그 개수를 반환함
s.index(item,[시작],[종료])	• 리스트에서 항목 item을 찾아 가장 작은 인덱스를 반환함 • 탐색의 시작 위치와 종료 위치를 지정할 수도 있음
s.insert(pos, item)	• pos 위치에 항목 item을 삽입함
s.pop(pos)	• pos 위치의 항목을 s에서 꺼내고 반환함
s.remove(item)	• 항목 item을 s에서 제거함
s.reverse()	• 리스트 항목의 순서를 뒤집음
s.sort([key],[reverse])	• 항목을 정렬함

컬렉션 자료형의 활용

튜플(tuple)

리스트와 동일하나 크기가 고정됨

메모리 관리가 효율적임

```
t = ( 0, 3, 7 )                                  # 항목이 3개인 튜플
a = ( 2 )                                        # 항목이 1개인 튜플
b = ( 'game', 1, 3.14, 2019)                     # 항목이 4개인 복합 튜플
```

```
print("취미=%s, 나이=%d, 학점=%f" % (hobby, age, score))          # 튜플
```

컬렉션 자료형의 활용

딕셔너리(dict)

키(Key)와 관련된 값(Value)로 이루어진 항목(Entry)들의 집합

```
map = {'김연아':'피겨', '류현진':'야구', '쿠드롱':'당구', '메시':'축구'}
print(map)
print('쿠드롱이 뭐하는 사람이지?', map['쿠드롱'])
```

C:\WINDOWS\system32\cmd.exe

```
{'김연아':'피겨', '류현진':'야구', '쿠드롱':'당구', '메시':'축구'}
쿠드롱이 뭐하는 사람이지? 당구
```

컬렉션 자료형의 활용

딕셔너리(dict)

```
map['나달'] = '테니스'                                  # 맵에 항목 추가
map.update({'최민영':'여자야구', '고진영':'골프'})       # 여러 항목 추가
```

C:\WINDOWS\system32\cmd.exe

```
{'김연아':'피겨', '류현진':'야구', '쿠드롱':'당구', '메시':'축구',
'나달':'테니스', '최민영':'여자야구', '고진영':'골프'}
```

컬렉션 자료형의 활용

집합(set)

set과 frozenset(내용을 변경할 수 없는 set)이 있음

```
s1 = { 1,2,3 }                # 집합 객체
s2 = { 2,3,4,5 }              # 집합 객체
s3 = s1.union(s2)             # 합집합
s4 = s1.intersection(s2)      # 교집합
s5 = s1 - s2                  # 차집합
print("s1:", s1)
print("s2:", s2)
print("s3:", s3)
print("s4:", s4)
print("s5:", s5)
```

C:\WINDOWS\system32\cmd.exe

```
s1: {1, 2, 3}
s2: {2, 3, 4, 5}
s3: {1, 2, 3, 4, 5}
s4: {2, 3}
s5: {1}
```

컬렉션 자료형의 활용

집합(set)

```
s5 = { 3.14 }                    # 원소가 하나인 집합
map = { 3.14 : 'Phi' }           # 엔트리가 하나인 딕셔너리
```

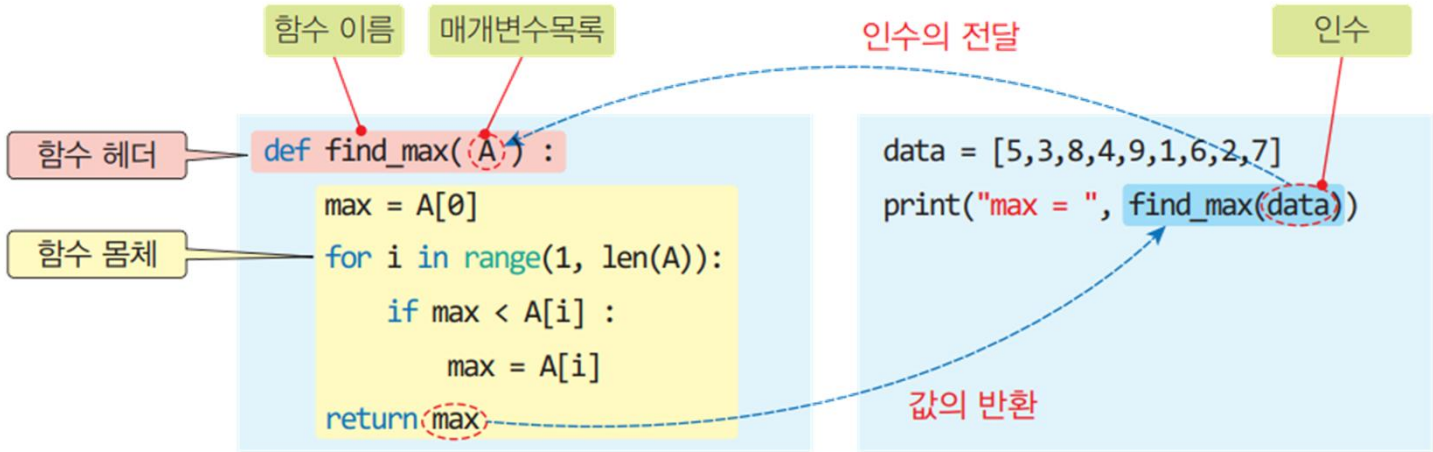
함수, 변수, 모듈

함수, 변수, 모듈의 활용

사용자 정의 함수와 내장 함수

사용자 정의 함수 ⟷ 파이썬 내장 함수

- type(), len(), ord() 등



함수의 정의 함수의 호출

함수, 변수, 모듈의 활용

함수값 반환

여러 값 반환

```
def find_min_max(A) :                          # 최댓값과 최솟값을 동시에 찾아 반환
    min = A[0]
    max = A[0]
    for i in range(1, len(A)) :                # i : 1 ~ len(A)-1
        if max < A[i] : max = A[i]             # 최댓값 갱신
        if min > A[i] : min = A[i]             # 최솟값 갱신
    return min, max                            # 최솟값과 최댓값을 반환
```

```
data = [ 5, 3, 8, 4, 9, 1, 6, 2, 7 ]
x, y = find_min_max(data)                      # 최솟값과 최댓값을 반환받음
print("(min,max)", (x,y))                      # x와 y를 튜플로 만들어 출력
```

함수, 변수, 모듈의 활용

함수 인수 설정

Default 인수(초기설정)

```
def sum_range(begin, end, step=1) :                # 매개변수 step이 기본값을 가짐
    sum = 0
    for n in range(begin, end, step) :
        sum += n
    return sum
```

```
print("sum = ", sum_range(1, 10))                  # step은 디폴드 값(1)으로 처리됨
```

```
print("sum = ", sum_range(1, 10, 2))               # 정상 호출, step은 2
```

함수, 변수, 모듈의 활용

함수 인수 설정

Keyword 인수

```
print("sum = ", sum_range(step=3, begin=1, end=10))    # 키워드 인수 사용
```

```
print("game", end="")                # 라인피드가 발생하지 않음(키워드 인수 사용)
```

함수, 변수, 모듈의 활용

변수의 범위

내장 범위 (Built-in Scope)	• 언어의 일부로 정의된 변수와 리터럴들 • 프로그램의 어디에서나 사용할 수 있음
전역 범위 (Global Scope)	• 소스 파일의 맨 꼭대기 레벨(함수나 클래스 밖)에서 생성 • 프로그램의 어디에서나 사용할 수 있음
지역 범위 (Local Scope)	• 함수나 클래스의 멤버함수(메소드) 안에서 생성 • 그 안에서만 사용함 • 함수의 매개변수들도 지역범위임
인스턴스 범위 (Instance Scope)	• 클래스의 데이터 멤버로 생성된 변수 • 클래스 내의 다른 함수들에서 사용할 수 있음

함수, 변수, 모듈의 활용

모듈과 Namespace

```
# 파일명: min_max.py                          # 다음 함수가 min_max.py에 저장되어 있음
def find_min_max(A) :                         # 최댓값과 최솟값을 동시에 찾아 반환
    …                                         # 2.7장의 find_min_max()함수 코드와 동일
    return min, max                           # 최솟값과 최댓값을 반환
```

```
# 파일명: sum.py                              # 다음 함수가 sum.py에 저장되어 있음
def sum_range(begin, end, step=1) :           # 매개변수 step이 기본 값을 가짐
    …                                         # 2.7장의 sum_range()함수 코드와 동일
    return sum
```

함수, 변수, 모듈의 활용

모듈과 Namespace

```
# 파일명: my_job.py
import min_max                                          # min_max.py 모듈을 사용함
import sum                                              # sum.py 모듈을 사용함

data = [ 5, 3, 8, 4, 9, 1, 6, 2, 7 ]
print("(min,max) = ", min_max.find_min_max(data))       # min_max 모듈의 함수 사용
print("sum = ", sum.sum_range(1, 10))                   # sum 모듈의 함수 사용
```

함수, 변수, 모듈의 활용

모듈과 Namespace

```
from math import pow, sqrt                    # math 모듈에서 pow, sqrt 식별자를 사용
result = pow(2,10)                            # pow 식별자를 바로 사용 가능
dist = sqrt(1000)                             # sqrt 식별자를 바로 사용가능
```

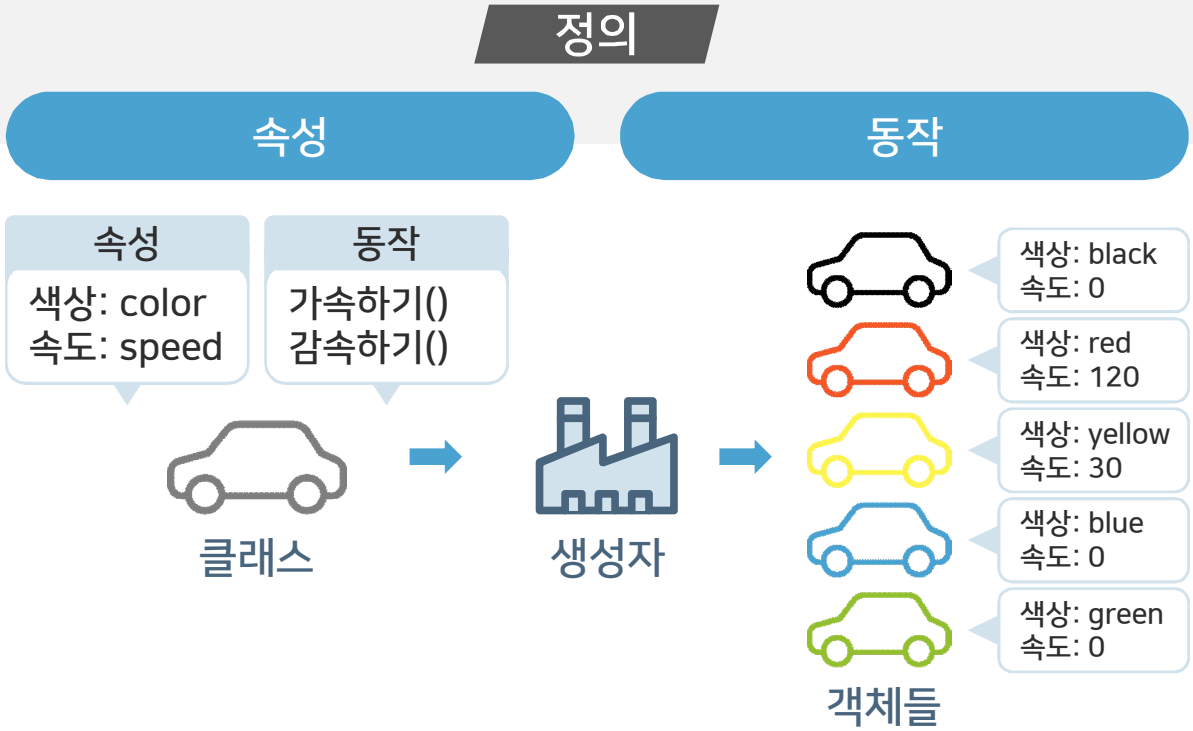
```
# 파일명: my_job.py
from min_max import *                         # min_max  모듈의 모든 식별자 사용 가능
from sum import *                             # sum  모듈의 모든 식별자 사용 가능

data = [ 5, 3, 8, 4, 9, 1, 6, 2, 7 ]
print("(min,max) = ", find_min_max)           # 바로 사용
print("sum = ", sum_range(1, 10))             # 바로 사용
```

객체지향 프로그래밍

객체지향 프로그래밍의 활용

클래스




```
class Car :      # 자동차와 관련된 클래서 정의 블록이 이어짐. 들여쓰기에 유의할 것
```

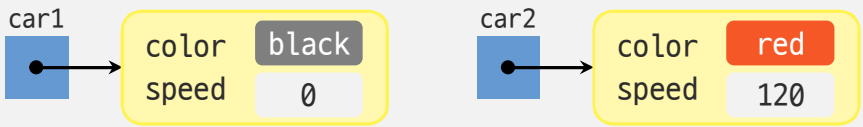
객체지향 프로그래밍의 활용

클래스

생성자

```
class Car :
    def __init__(self, color, speed = 0)        # 생성자 함수
        self.color = color                      # 데이터 멤버 color 정의 및 초기화
        self.speed = speed                      # 데이터 멤버 speed 정의 및 초기화
```

```
car1 = Car('black', 0)          # 검정색, 속도 0
car2 = Car('red', 120)          # 빨간색, 속도 120
car3 = Car('yellow', 30)        # 노란색, 속도 30
car4 = Car('blue', 0)           # 파랑색, 속도 0
car5 = Car('green')             # 초록색, 속도 0(디폴트 인수 사용)
```



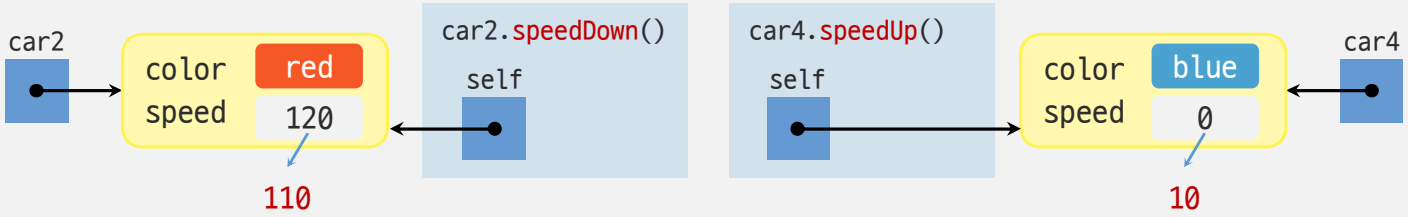
객체지향 프로그래밍의 활용

클래스

멤버 함수

```
class Car :
    # ...
    def speedUp(self) : self.speed += 10                # 가속 동작: 속도 10 증가
    def speedDown(self) : self.speed -= 10              # 감속 동작: 속도 10 감소
```

```
car2.speedDown()                # car2 감속: 속도 10 감소
car4.speedup()                  # car4 가속: 속도 10 증가
```



객체지향 프로그래밍의 활용

클래스

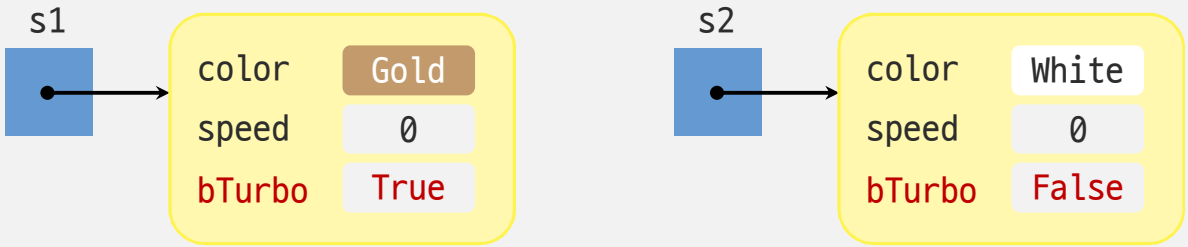
```
car3.color = 'purple'                    # car3의 색상을 purple로 변경
car5.speed = 100                         # car5의 속도를 100으로 변경
```

객체지향 프로그래밍의 활용

상속

```
class SuperCar(Car) :
    def __init__(self, color, speed = 0, bTurbo = True) :
        super().__init__(color, speed)                    # 부모(Car)클래스의 생성자 호출
        self.bTurbo = bTurbo                              # 터보모드를 위한 변수 생성 및 초기화
```

```
s1 = SuperCar("Gold", 0, True)
s2 = SuperCar("White", 0, False)
```



객체지향 프로그래밍의 활용

상속

```
s1.speedUp()
s2.speedUp()
print("슈퍼카1:", s1)
print("슈퍼카2:", s2)
```

C:\WINDOWS\system32\cmd.exe

```
슈퍼카1: color = Gold, speed = 10
슈퍼카2: color = White, speed = 10
```

객체지향 프로그래밍의 활용

재정의(Overriding)

```
def setTurbo(self, bTurbo = True) :                    # 터보모드를 On/Off하는 메소드
    self.bTurbo = bTurbo
```

```
def speedUp(self) :                                    # SuperCar의 SpeedUp. 메소드 재정의
    if self.bTurbo :                                   # 터보모드이면
        self.speed += 50                               # 속도가 급속히(50) 증가됨
    else :                                             # 터보모드가 아니면
        super().speedUp()                              # 일반 자동차의 가속 메소드 호출
```

```
def __str__(self) :
    if self.bTurbo :
        return "[%s] [speed = %d] 터보모드" % (self.color, self.speed)
    else :
        return "[%s] [speed = %d] 일반모드" % (self.color, self.speed)
```

객체지향 프로그래밍의 활용

재정의(Overriding)

```
s1.speedUp()
s2.speedUp()
print("슈퍼카1:", s1)
print("슈퍼카2:", s2)
```

C:\WINDOWS\system32\cmd.exe

```
슈퍼카1: [Gold] [speed = 50] 터보모드
슈퍼카2: [White] [speed = 10] 일반모드
```

객체지향 프로그래밍의 활용

연산자 중복

비교 연산자 == 중복

```
def __eq__(self, carB) : return self.color == carB.color
```

```
print("car2==car6 : ", car2==car6)
print("car3==car6 : ", car3==car6)
```

C:\WINDOWS\system32\cmd.exe

```
car2==car6 : False
car3==car6 : True
```

객체지향 프로그래밍의 활용

연산자 중복

문자열 변환 연산자 __str__

```
def __str__(self) :
    return "color = %s, speed = %d" % (self.color, self.speed)
```

```
print("[car3]", car3)
```

C:\WINDOWS\system32\cmd.exe

```
[car3] color = yellow, speed = 30
```

학습평가

학습한 내용을 바탕으로 다음 문제를 풀어보세요.

번호	문제	보기	

1	아래와 같이 파이썬 변수를 선언했을 때, 변수 value의 출력값은 무엇인가요?
	`number = 1023`
	`score = 199`
	`value = number`
	`number =717`
	`print(value)`
		① 1023
		② 199
		③ 717
		④ null	

2	아래의 파이썬 프로그램의 출력값은 무엇인가요?
	`mySet = [1, 3, 5, 7, 9]`
	`sum =0`
	`for e in mySet :`
	`  sum = sum + e`
	`print(sum/4, sum//4, sum%4`
		① 6 1 6.25
		② 6.25 6 1
		③ 6.25 1 6
		④ 6 6.25 1	

3	a=[11, 20, 30, 40, 50, 60] 에서 인덱스로 요소를 가져올 때 값이 올바르지 않은 것은 무엇인가요?	① a[0]은 20 ② a[1]은 20 ③ a[3]은 40 ④ a[-1]은 60	
4	튜플 tp=(10, 22, 33, 44, 50, 66, 77, 88, 90)에서 (10, 50, 90)을 가져오는 방법으로 올바른 것은 무엇인가요?	① tp[0:9] ② tp[0:9:3] ③ tp[: : 4] ④ tp[0:6:2]	

학습평가

학습한 내용을 바탕으로 다음 문제를 풀어보세요.

번호	문제	보기	
5	매개변수가 없는 say_hello 함수를 호출하는 방법은 무엇인가요?	① def say_hello ② say_hello ③ say_hello[] ④ say_hello()	
6	두수를 곱한 결과를 반환하는 함수는 무엇인가요?	① def multip() : a * b ② def multip(a, b) : a * b ③ multip(a, b) : return a * b ④ def multip(a, b) : return a * b	
7	다음 클래스의 greeting 메소드를 호출하는 방법은 무엇인가요? class Persons : def greeting(self) : print('hello')	① john = Persons() john.greeting() ② Persons.greetin() ③ john = Persons john.greeting() ④ greeting()	
8	클래스로 인스턴스를 만들 때 호출되는 메소드는 __init__ 이다.	① O ② X	

학습정리

파이썬의 이론적 기초

파이썬 언어의 기초

- 파이썬에서는 모든 자료가 클래스로부터 만들어진 객체이다.
- 변수는 다른 객체를 참조하는 참조자 또는 포인터의 역할을 한다.
- 나눗셈 연산자, 단항 연산자, 불리언 연산자 등에서 파이썬 만의 특징이 존재한다.
- 제어구조와 반복 관련한 구문에는 if, else, elif, for, while 등이 있다.
- 입출력 함수로는 input, print 등이 있다.

컬렉션 자료형

- 컬렉션 자료형으로 str, list, tuple, dict, set 등이 존재한다.

함수, 변수, 모듈

- 사용자 정의 함수를 정의하고 호출할 수 있다.
- 변수는 효과 범위에 따라 내장 범위, 전역 범위, 지역 범위, 인스턴스 범위 등으로 분류한다.

객체지향 프로그래밍

- 클래스는 속성을 나타내는 멤버 변수와 동작을 나타내는 메소드로 구성된다.
- 모든 클래스는 특별한 멤버 함수인 생성자를 정의해야 한다.
- 상속은 기존 정의된 클래스로부터 멤버를 추가해 새로운 클래스를 만드는 것이다.